PANDUAN MEDICHAIN

Simulasi MockChain dan perpindahan ke Ethereum Sepolia

Tujuan. Backend sudah memiliki dua implementasi chain: `MockChain` untuk simulasi lokal dan `Web3Chain` untuk kontrak Sepolia. Pemilihan dilakukan lewat satu nilai di file `.env`; tidak perlu mengomentari atau menukar baris kode untuk memilih jaringan.

Pahami pemisahan data

Bagian	Lokasi / dampak
Data aplikasi	PostgreSQL. Mengganti mode chain tidak menghapus atau memindahkan data SQL.
MockChain	Ledger lokal: `.mockchain.json` atau volume Docker `/data/mockchain.json`. Tetap terpisah dari Sepolia.
Sepolia	Jaringan publik. Hash/key pseudonim, versi, waktu, event akses, dan transaksi terlihat publik; transaksi membayar gas dengan Sepolia test ETH.

Tahap 1 ? jalankan simulasi MockChain

- Pastikan `CHAIN_MODE=mock` di file `.env`. Mode ini tidak memerlukan RPC, kontrak, private key, atau ETH.
- Jalankan aplikasi seperti biasa. Docker: `docker compose up --build`. Untuk lokal, ikuti bagian menjalankan aplikasi di `README.md`.
- Gunakan data sintetis, buat atau ubah rekam, lalu buka menu **Integritas**. Periksa status valid, versi, dan transaksi mock. Admin/auditor dapat menjalankan sinkronisasi jika antrean masih pending.
- Hentikan backend sebelum mengganti mode. Database tetap utuh; ledger mock juga tetap ada.

Tahap 2 ? siapkan Sepolia

- Buat wallet khusus testnet. Jangan pernah memakai atau menempelkan private key mainnet. Isi test ETH dari faucet Sepolia.
- Deploy `contracts/MediChain.sol` (Solidity 0.8.20+) ke Sepolia dengan MetaMask. Catat alamat kontraknya. Chain ID harus `11155111`.
- Akun backend harus berstatus writer. Gunakan akun deployer atau panggil `setWriter(backendAddress, true)` dari owner kontrak.
- Di `.env`, ubah `CHAIN_MODE=mock` menjadi `CHAIN_MODE=web3`, lalu isi RPC, alamat kontrak, dan private key Sepolia. Jangan menimpa seluruh .env dan jangan membagikan isinya.

```
CHAIN_MODE=web3
CHAIN_ID=11155111
CHAIN_NAME=Sepolia
RPC_URL=https://RPC-SEPOLIA-ANDA
CONTRACT_ADDRESS=0xALAMAT_KONTRAK
CHAIN_PRIVATE_KEY=0xPRIVATE_KEY_TESTNET
```

Template lengkap ada di `.env.sepolia.example`. Jangan commit file `.env`. Periksa juga bahwa URL RPC menunjuk ke Sepolia.

Tahap 3 ? periksa sebelum mengirim

• Buat backup PostgreSQL dan pastikan file backup dapat dibaca. Jangan menghapus volume database atau file ledger mock.

• Restart/rebuild layanan agar backend memuat konfigurasi baru: `docker compose up --build -d`. Ubah mode saat backend berhenti.

• Jika database memiliki rekam yang sudah dibuat di MockChain, jalankan dry-run terlebih dahulu. Perintah ini mengestimasi biaya dan tidak menulis ke SQL maupun blockchain:

```
docker compose exec api python -m scripts.anchor_existing_records --dry-run
```

Periksa jaringan, jumlah rekam, estimasi gas, dan saldo akun. Jika kontrak sudah memiliki rekam dengan hash atau versi berbeda, proses akan berhenti. Jangan mencoba mengubah atau menimpa hash tersebut.

Tahap 4 ? kirim transaksi uji

• Jika database sudah berisi rekam lama dan dry-run lolos, perintah berikut mengantrekan lalu mengirim snapshot aktif serta izin akses ke Sepolia. Ini transaksi publik dan membutuhkan gas:

```
docker compose exec api python -m scripts.anchor_existing_records
```

• Jika database kosong, lewati anchor. Buat satu pasien sintetis dari aplikasi setelah memastikan CHAIN_MODE=web3.

• Buka menu **Integritas**. Cocokkan jaringan, versi, status hash, dan tautan transaksi Etherscan Sepolia. Transaksi tertunda bisa dicoba lagi lewat **Sinkronkan transaksi**.

• Data klinis mentah tetap di database; kontrak menyimpan hash dan metadata. Hash dan pola transaksi tetap publik. Kontrak ini prototipe dan belum diaudit pihak ketiga.

Kembali ke MockChain

Hentikan backend, set `CHAIN_MODE=mock`, lalu mulai ulang layanan. Data PostgreSQL dan ledger MockChain tetap tersedia. Aktivitas yang sudah terkirim ke Sepolia tidak terhapus, dan tidak otomatis disalin ke ledger mock.

Jika file Python tampak merah di VS Code

Sering kali garis merah berasal dari interpreter yang tidak memiliki dependensi proyek. Pilih Python environment yang dipakai backend, lalu pasang dependensi di environment tersebut:

```
cd medichain-app
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements-dev.txt
python -m pytest -q
```

Di VS Code tekan **Ctrl+Shift+P** ? **Python: Select Interpreter** ? pilih `medichain-app\.venv`. Buka ulang folder `medichain-app` agar import lokal `app.*` terdeteksi. Jangan jalankan tes pada data produksi; suite proyek mengarahkan tes ke SQLite dan MockChain sementara.

Pemeriksaan pada workspace ini: semua file Python di `app`, `scripts`, dan `migrations` berhasil dikompilasi. Tes belum dijalankan karena interpreter aktif belum memasang `pytest`.